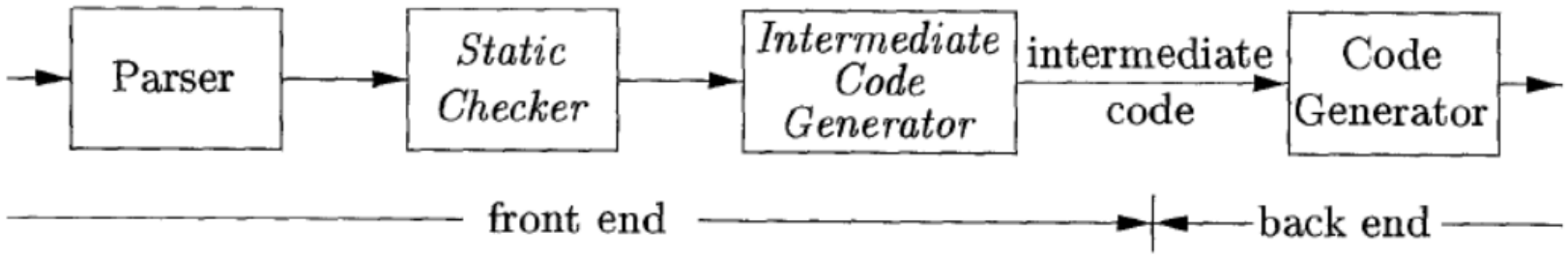
Intermediate-Code Generation

Intermediate-Code Generation



Three-Address Code

- In three-address code, there is at **most one operator** on the **right side of an instruction**
- Thus a **source-language expression like x+y*z** might be translated into the sequence of three-address instructions

$$t_1 = y * z$$
$$t_2 = x + t_1$$

Common three-address instructions

1. Assignment instructions of the form $x = y \ op \ z$, where op is a binary arithmetic or logical operation, and x, y, and z are addresses.

2. Assignments of the form $x = op \ y$, where op is a unary operation. Essential unary operations include unary minus, logical negation, shift operators, and conversion operators that, for example, convert an integer to a floating-point number.

3. *Copy instructions* of the form $x = y$, where x is assigned the value of y.

4. An unconditional jump `goto` L. The three-address instruction with label L is the next to be executed.

5. Conditional jumps of the form `if` x `goto` L and `ifFalse` x `goto` L. These instructions execute the instruction with label L next if x is true and false, respectively. Otherwise, the following three-address instruction in sequence is executed next, as usual.

6. Conditional jumps such as `if` x *relop* y `goto` L, which apply a relational operator (<, ==, >=, etc.) to x and y, and execute the instruction with label L next if x stands in relation *relop* to y. If not, the three-address instruction following `if` x *relop* y `goto` L is executed next, in sequence.

7. Procedure calls and returns are implemented using the following instructions: `param` x for parameters; `call` p, n and y = `call` p, n for procedure and function calls, respectively; and `return` y, where y, representing a returned value, is optional. Their typical use is as the sequence of three-address instructions

```
param x1
param x2
...
param xn
call p, n
```

generated as part of a call of the procedure $p(x_1, x_2, \ldots, x_n)$. The integer n, indicating the number of actual parameters in "`call` p, n,"

8. Indexed copy instructions of the form $x = y[i]$ and $x[i] = y$. The instruction $x = y[i]$ sets x to the value in the location i memory units beyond location y. The instruction $x[i] = y$ sets the contents of the location i units beyond x to the value of y.

Common three-address instructions

```
do i = i+1; while (a[i] < v);
```

```
L:  t1 = i + 1
    i = t1
    t2 = i * 8
    t3 = a [ t2 ]
    if t3 < v goto L
```

Data structures for TAC

Quadruples

A *quadruple* (or just "*quad*") has four fields, which we call *op*, arg_1, arg_2, and *result*. The *op* field contains an internal code for the operator. For instance, the three-address instruction $x = y + z$ is represented by placing + in *op*, y in arg_1, z in arg_2, and x in *result*. The following are some exceptions to this rule:

1. Instructions with unary operators like x = `minus` y or $x = y$ do not use arg_2. Note that for a copy statement like $x = y$, *op* is =, while for most other operations, the assignment operator is implied.

2. Operators like `param` use neither arg_2 nor *result*.

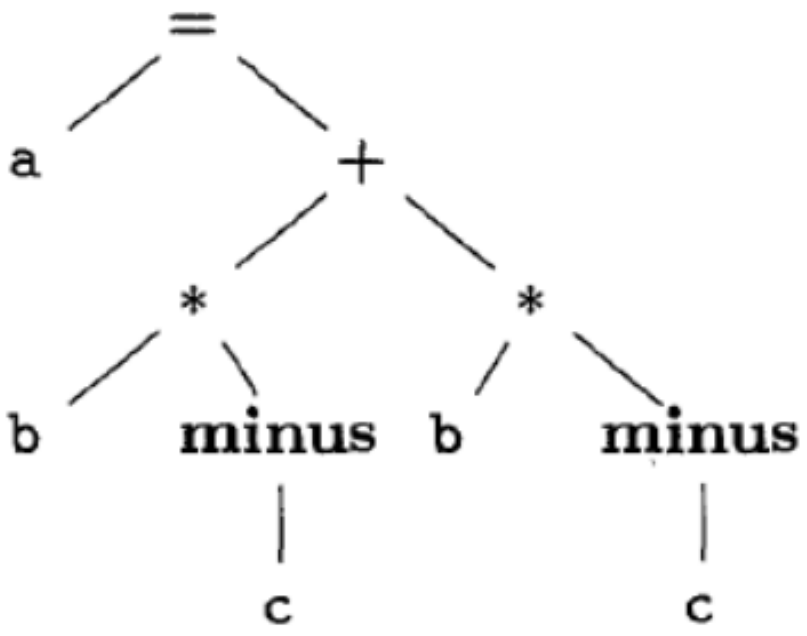
3. Conditional and unconditional jumps put the target label in *result*.

Data structures for TAC
Quadruples

Three-address code for the assignment `a = b*-c+b*-c;`

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Three-address code



(a) Syntax tree

Data structures for TAC
Quadruples

Three-address code for the assignment `a = b*-c+b*-c;`

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Three-address code

	op	arg_1	arg_2	*result*
0	minus	c		t_1
1	*	b	t_1	t_2
2	minus	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

(b) Quadruples

Data structures for TAC
Triples

- A triple has **only three fields**, which we call **op, arg1, and arg2**
- Note that the **result field in Quad** is used primarily for temporary names.
- Using **triples**, we refer to the **result of an operation** x op y **by its position**, rather than by an explicit temporary name.
- Thus, instead of the **temporary t**, a triple representation would refer to **position (0)**.
- **Parenthesized numbers** represent pointers into the **triple structure** itself.

Data structures for TAC
Triples

Three-address code for the assignment `a = b*-c+b*-c;`

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Three-address code

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

(b) Triples

Benefit of Quad over Triples

- A benefit of **quadruples** over triples can be seen in an **optimizing compiler**, where **instructions are often moved around**.
    - With quadruples, **if we move an instruction** that **computes a temporary t**, then the instructions that **use t** require **no change**.
- With **triples**, the **result** of an operation is referred to by **its position**
- So **moving an instruction** may require us to **change all references** to that result.

Indirect triples

- Consist of a **listing of pointers** to triples,
    - Rather than a listing of triples themselves.
- For example, use an **array *instruction*** to list **pointers to triples** in the desired order.

With indirect triples, an **optimizing compiler** can move an instruction by **reordering the instruction list**, without affecting the triples themselves.

instruction	
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

	op	arg_1	arg_2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

	op	arg_1	arg_2
0	minus	c	
1	minus	c	
2	*	b	(0)
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Static Single-Assignment Form

Two distinctive aspects distinguish SSA from three-address code.
(a) The first is that **all assignments in SSA** are to **variables with distinct names**; hence the term static single-assignment.

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

(a) Three-address code.

$$
\begin{aligned}
p_1 &= a + b \\
q_1 &= p_1 - c \\
p_2 &= q_1 * d \\
p_3 &= e - p_2 \\
q_2 &= p_3 + q_1
\end{aligned}
$$

(b) Static single-assignment form.

Static Single-Assignment Form

Two distinctive aspects distinguish SSA from three-address code.
(a) The first is that **all assignments in SSA** are to **variables with distinct names**; hence the term static single-assignment.
(b)

The same variable may be defined in two different control-flow paths in a program. For example, the source program

```
if ( flag ) x = -1; else x = 1;
y = x * a;
```

has two control-flow paths in which the variable x gets defined. If we use different names for x in the true part and the false part of the conditional statement, then which name should we use in the assignment y = x * a? Here is where the second distinctive aspect of SSA comes into play. SSA uses a notational convention called the ϕ-function to combine the two definitions of x:

```
if ( flag ) x1 = -1; else x2 = 1;
x3 = φ(x1, x2);
```

Static Single-Assignment Form

Here, $\phi(\mathbf{x}_1, \mathbf{x}_2)$ has the value $\mathbf{x}_1$ if the control flow passes through the true part of the conditional and the value $\mathbf{x}_2$ if the control flow passes through the false part. That is to say, the ϕ-function returns the value of its argument that corresponds to the control-flow path that was taken to get to the assignment-statement containing the ϕ-function.

Major translation classes of Three address code generation

(a) Declaration statements (+ handling data type and storage)

(b) Expressions and statements

(a) Control flow statements

Declaration statement

Representing data types: **Type Expressions**

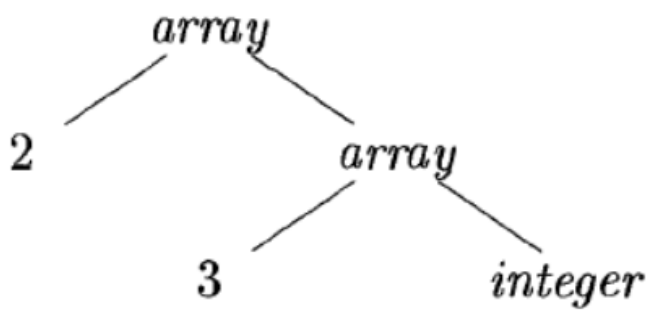
Types have structure, which we shall represent using type expressions.

- A **type expression is either a basic type** (*boolean, char, integer, float,* and *void*)

  or
- is formed by **applying an operator** called a **type constructor** to a type expression.
- A **type expression** can be formed by applying the **array type constructor** to a **number and a type expression**.

Declaration statement

- The array type int [2] [3] can be read as "array of 2 arrays of 3 integers each"
- Can be represented as a **type expression** array(2, array(3, integer)).
- This type is represented by the tree.



- The **operator array** takes **two parameters**, a number and a type.
  - Here the **type expression** can be formed by applying the **array type constructor** to a number and a type expression.

Declaration statement Example SDD

PRODUCTION	SEMANTIC RULES
$T \rightarrow B\ C$	$T.t = C.t$
	$C.b = B.t$
$B \rightarrow \textbf{int}$	$B.t = integer$
$B \rightarrow \textbf{float}$	$B.t = float$
$C \rightarrow [\ \textbf{num}\ ]\ C_1$	$C.t = array(\textbf{num}.val,\ C_1.t)$
	$C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

Type Expressions

- Nonterminal T generates either a **basic type** or an **array type**.
- Nonterminal B generates one of the basic types int and float.
- T generates a basic type when C derives €.
- Otherwise, C generates array components consisting of a sequence of integers, each integer surrounded by brackets.

Declaration statement Example SDD

PRODUCTION	SEMANTIC RULES
$T \rightarrow B\ C$	$T.t = C.t$
	$C.b = B.t$
$B \rightarrow \textbf{int}$	$B.t = integer$
$B \rightarrow \textbf{float}$	$B.t = float$
$C \rightarrow [\ \textbf{num}\ ]\ C_1$	$C.t = array(\textbf{num}.val,\ C_1.t)$
	$C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

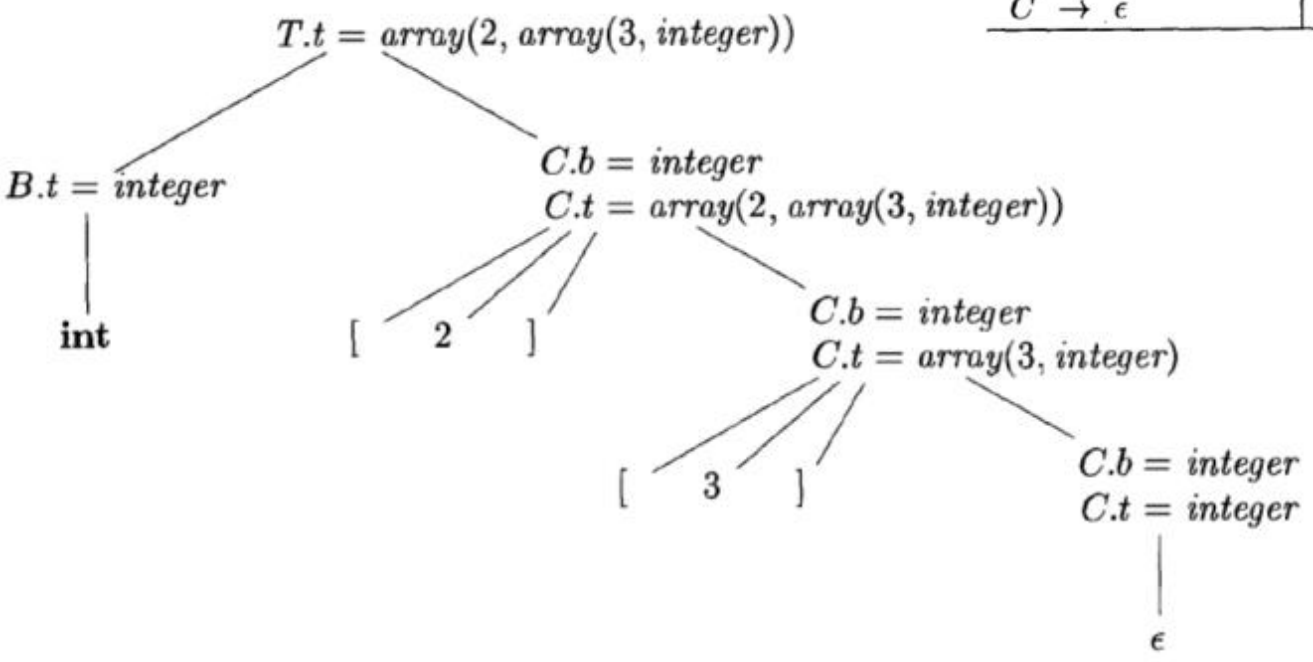
Type Expressions

- The nonterminals *B* and *T* have a synthesized attribute *t* representing a type.
- The nonterminal *C* has two attributes: an inherited attribute *b* and a synthesized attribute *t.*

Declaration statement Example SDD

PRODUCTION	SEMANTIC RULES
$T \rightarrow B\ C$	$T.t = C.t$
	$C.b = B.t$
$B \rightarrow \textbf{int}$	$B.t = integer$
$B \rightarrow \textbf{float}$	$B.t = float$
$C \rightarrow [\ \textbf{num}\ ]\ C_1$	$C.t = array(\textbf{num}.val,\ C_1.t)$
	$C_1.b = C.b$
$C \rightarrow \epsilon$	$C.t = C.b$

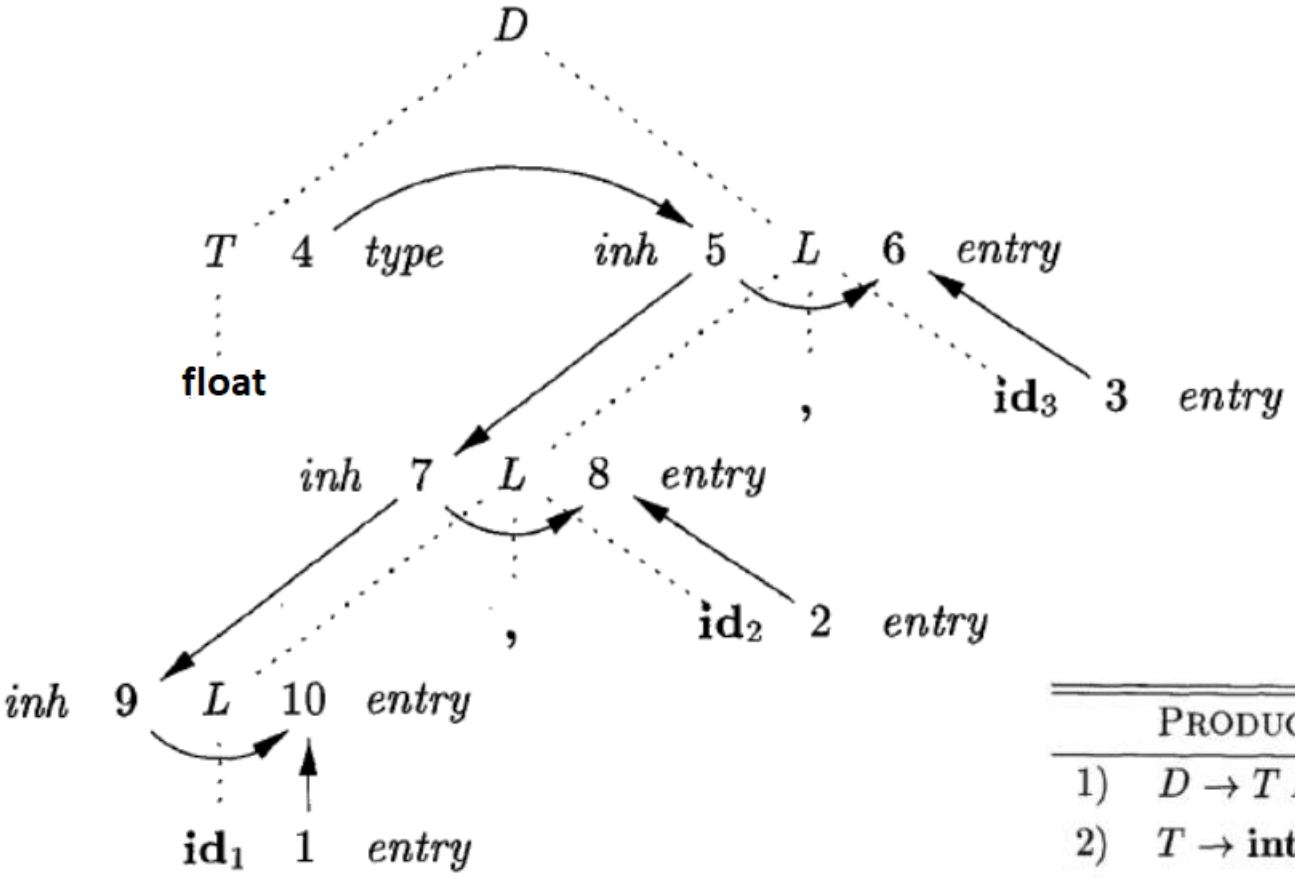
input string int [2][3]



- The nonterminal *C* has two attributes: an inherited attribute *b* and a synthesized attribute *t.*
- The inherited *b* attributes pass a basic type down the tree, and the synthesized *t* attributes accumulate the result.

Declaration statement: Example SDD

float $\mathbf{id}_1$, $\mathbf{id}_2$, $\mathbf{id}_3$



	PRODUCTION	SEMANTIC RULES
1)	$D \rightarrow T\ L$	$L.inh = T.type$
2)	$T \rightarrow \mathbf{int}$	$T.type = \text{integer}$
3)	$T \rightarrow \mathbf{float}$	$T.type = \text{float}$
4)	$L \rightarrow L_1\ ,\ \mathbf{id}$	$L_1.inh = L.inh$
		$addType(\mathbf{id}.entry, L.inh)$
5)	$L \rightarrow \mathbf{id}$	$addType(\mathbf{id}.entry, L.inh)$

Symbol table

ST(global) *This is the Symbol Table for global symbols*

Name	Type	Initial Value	Size	Offset	Nested Table
d	float	2.3	8	0	null
i	int	null	4	8	null
w	*array*(10, int)	null	40	12	null
a	int	4	4	52	null
p	*ptr*(int)	null	4	56	null
b	int	null	4	60	null
func	*function*	null	0	64	ptr-to-ST(func)
c	char	null	1	64	null

Find the storage for each variable

More on Declaration statement

Data type + Storage layout

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

- Simplified grammar that declares just **one name at a time**;
- We already explored the declarations with lists of names

Storage layout:

- **Relative address** of all the variables
- From the **type of a variable**, we can determine the **amount of storage** that will be needed for the variable at run time.
- At **compile time**, we can use these amounts to **assign each variable a relative address**.
- The **type** and **relative address** are saved in the **symbol-table entry** for the variable name.

More on Declaration statement

Data type + Storage layout

- The **width** of a **type** is the number of **storage units** needed for objects of that type **(offset)**.
- A **basic type**, such as a character, integer, or float, requires an integral number of bytes.
- **Arrays** allocated in one **contiguous block of bytes**

$T \rightarrow B \; C$	$\{ t = B.type; \; w = B.width; \}$
$B \rightarrow \textbf{int}$	$\{ B.type = integer; \; B.width = 4; \}$
$B \rightarrow \textbf{float}$	$\{ B.type = float; \; B.width = 8; \}$
$C \rightarrow \epsilon$	$\{ C.type = t; \; C.width = w; \}$
$C \rightarrow [\; \textbf{num} \;] \; C_1$	$\{ array(\textbf{num}.value, \; C_1.type); \; C.width = \textbf{num}.value \times C_1.width; \}$

Computes **data types** and their **widths** for basic and array types

Type Expressions

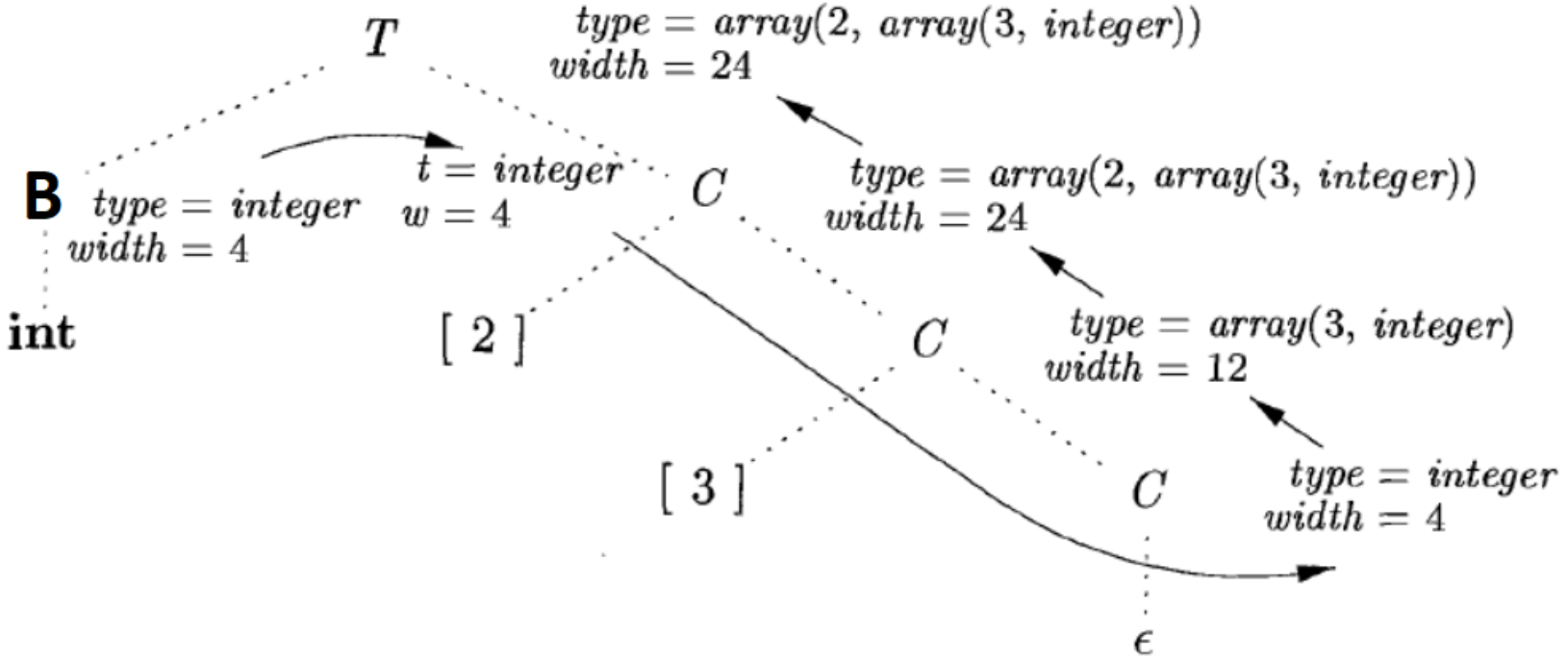
The **width of an array** is obtained by **multiplying** the **width of an element** by the **number of elements** in the array.

More on Declaration statement

Data type + Storage layout

```
int[2][3]
```

$T \rightarrow B$		$\{ t = B.type;\ w = B.width; \}$
$\quad\ \ C$		
$B \rightarrow$	**int**	$\{ B.type = integer;\ B.width = 4; \}$
$B \rightarrow$	**float**	$\{ B.type = float;\ B.width = 8; \}$
$C \rightarrow$	ϵ	$\{ C.type = t;\ C.width = w; \}$
$C \rightarrow$	[**num**] C_1	$\{ array(\mathbf{num}.value,\ C_1.type);$ $C.width = \mathbf{num}.value \times C_1.width; \}$



Relative address

Name	Data type	Relative address
d	float	0
i	int	8
w	*array*(10, int)	12

Sequences of Declarations

int x;
float y;

- Relative address: **offset**
- Keeps track of the **next available relative address**

$$P \rightarrow \{ \; offset = 0; \; \} \; D$$

$$D \rightarrow T \; \mathbf{id} \; ; \quad \{ \; top.put(\mathbf{id}.lexeme, \; T.type, \; offset); \; offset = offset + T.width; \; \} \; D_1$$

$$D \rightarrow \epsilon$$

- The translation scheme deals with a **sequence of declarations** of the form T id, where T generates a data type
- Before the first declaration is considered, **offset is set to 0**.
- As **each new name x** is seen, x is entered into the **symbol table** with its **relative address = current value of offset**,
    - which is **then incremented** by the width of the type of x.

Sequences of Declarations

- Relative address: offset
- Keeps track of the next available relative address

$$
\begin{aligned}
P &\rightarrow \{\ \mathit{offset} = 0;\ \}\ D \\
D &\rightarrow T\ \mathbf{id}\ ;\quad \{\ \mathit{top.put}(\mathbf{id}.\mathit{lexeme},\ T.\mathit{type},\ \mathit{offset});\\
&\qquad\qquad \mathit{offset} = \mathit{offset} + T.\mathit{width};\ \}\\
&\quad D_1 \\
D &\rightarrow \epsilon
\end{aligned}
$$

The semantic action within the production $D \rightarrow T\ \mathbf{id}\ ;\ D_1$ creates a symbol-table entry by executing $\mathit{top.put}(\mathbf{id}.\mathit{lexeme},\ T.\mathit{type},\ \mathit{offset})$. Here *top* denotes the current symbol table. The method *top.put* creates a symbol-table entry for $\mathbf{id}.\mathit{lexeme}$, with type $T.\mathit{type}$ and relative address *offset* in its data area.

Record or Structure data type

$$T \rightarrow \textbf{record} \ '\{' \ D \ '\}'$$

The fields in this record type are specified by the sequence of declarations generated by D. The approach of Fig. 6.17 can be used to determine the types and relative addresses of fields, provided we are careful about two things:

- The field names within a record must be distinct; that is, a name may appear at most once in the declarations generated by D.
- The offset or relative address for a field name is relative to the data area for that record.

```
float x;
record { float x; float y; } p;
record { int tag; float x; float y; } q;
```

```
x = p.x+q.x;
```

Record or Structure data type

$$T \rightarrow \textbf{record}\ '\{'\ D\ '\}'$$

$$\begin{array}{lll} T \rightarrow \textbf{record}\ '\{' & \{\ Env.push(top);\ top = \textbf{new}\ Env(); \\ & \quad Stack.push(offset);\ offset = 0;\ \} \\ \quad D\ '\}' & \{\ T.type = record(top);\ T.width = offset; \\ & \quad top = Env.pop();\ offset = Stack.pop();\ \} \end{array}$$

For convenience, record types will encode both the types and relative addresses of their fields, using a symbol table for the record type. A record type has the form *record*(t), where *record* is the type constructor, and t is a symbol-table object that holds information about the fields of this record type.

The embedded action before D saves the existing symbol table, denoted by *top* and sets *top* to a fresh symbol table. It also saves the current *offset*, and sets *offset* to 0. The declarations generated by D will result in types and relative addresses being put in the fresh symbol table. The action after D creates a record type using *top*, before restoring the saved symbol table and offset.

Record or Structure data type

$$T \rightarrow \textbf{record}\ '\{'\ D\ '\}'$$

$$\begin{array}{lll} T \rightarrow \textbf{record}\ '\{' & \{\ Env.push(top);\ top = \textbf{new}\ Env(); \\ & \quad Stack.push(offset);\ offset = 0;\ \} \\ \quad D\ '\}' & \{\ T.type = record(top);\ T.width = offset; \\ & \quad top = Env.pop();\ offset = Stack.pop();\ \} \end{array}$$

Let class *Env* implement symbol tables. The call *Env.push*(*top*) pushes the current symbol table denoted by *top* onto a stack. Variable *top* is then set to a new symbol table. Similarly, *offset* is pushed onto a stack called *Stack*. Variable *offset* is then set to 0.

After the declarations in D have been translated, the symbol table *top* holds the types and relative addresses of the fields in this record. Further, *offset* gives the storage needed for all the fields. The second action sets $T.type$ to *record*(*top*) and $T.width$ to *offset*. Variables *top* and *offset* are then restored to their pushed values to complete the translation of this record type.

Major translation classes of Three address code generation

(a) Declaration statements (+ handling data type and storage)

(b) Expressions and statements

(a) Control flow statements

Translation of Expressions

statement a = b + - c

Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ -E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid$ $gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

t_1 = minus c

t_2 = b + t_1

a = t_2

Attribute **code** for S

attributes **addr and code** for an expression E.

Attributes **S.code and E.code denote the three-address code** for S and E, respectively.

Attribute **E.addr denotes the address** that will hold the **value of E**.

Translation of Expressions

Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ -E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid$ $gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

When an expression is a **single identifier**, say x, then **x itself holds the value of the expression**.

The semantic rules for this production define **E.addr to point to the symbol-table entry**

Address of the variable

Translation of Expressions

Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \textbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\textbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \textbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$

The semantic rules for $E \rightarrow E_1 + E_2$, generate code to compute the value of E from the values of E_1 and E_2. Values are computed into newly generated temporary names. If E_1 is computed into $E_1.addr$ and E_2 into $E_2.addr$, then $E_1 + E_2$ translates into $t = E_1.addr + E_2.addr$, where t is a new temporary name. $E.addr$ is set to t. A sequence of distinct temporary names $t_1, t_2, \ldots$ is created by successively executing **new** $Temp()$.

Translation of Expressions

Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$

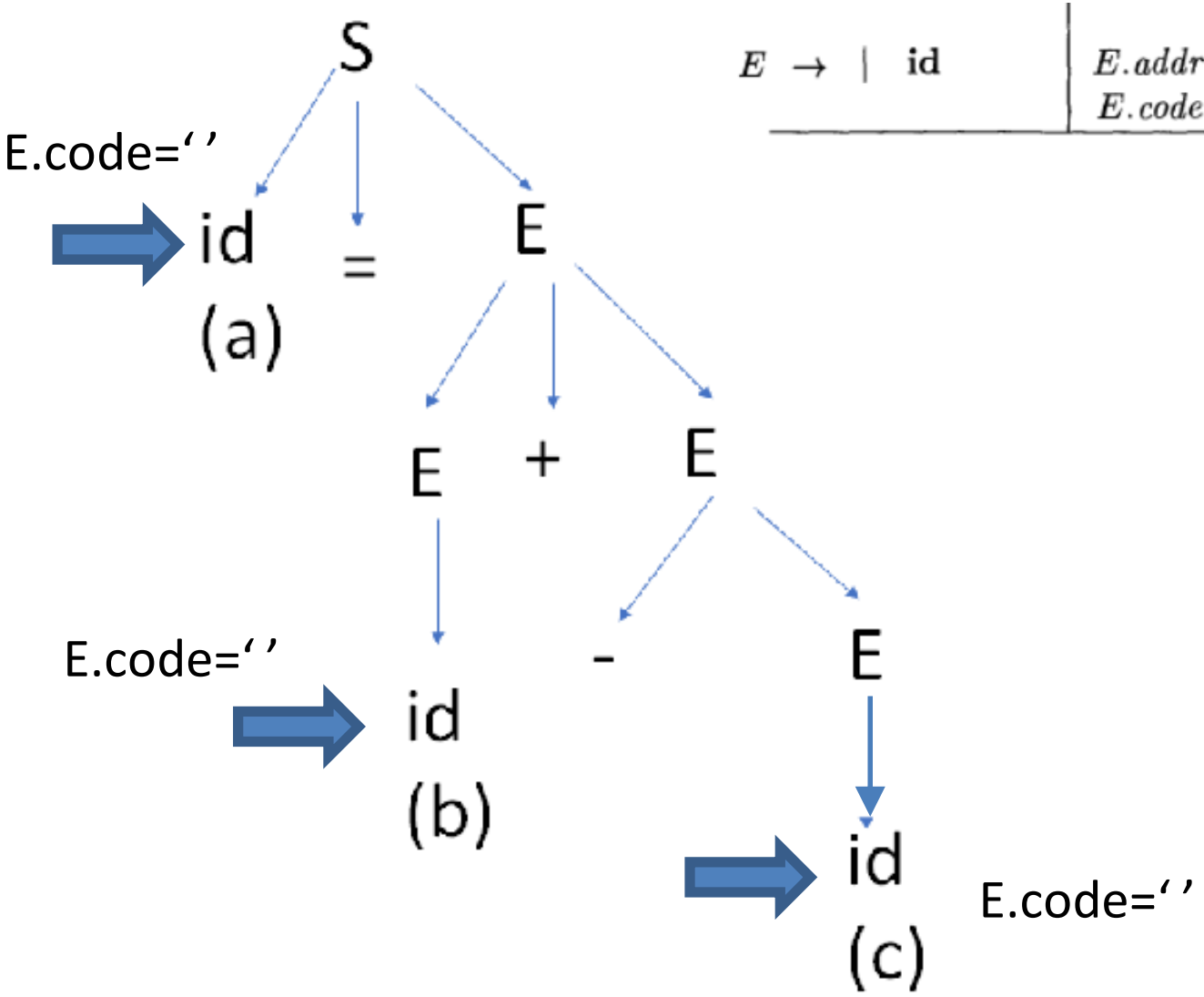
Finally, the production $S \rightarrow \mathbf{id} = E;$ generates instructions that assign the value of expression E to the identifier **id**. The semantic rule for this production uses function *top.get* to determine the address of the identifier represented by **id**, as in the rules for $E \rightarrow \mathbf{id}$. *S.code* consists of the instructions to compute the value of E into an address given by *E.addr*, followed by an assignment to the address *top.get*(**id**.*lexeme*) for this instance of **id**.

Translation of Expressions

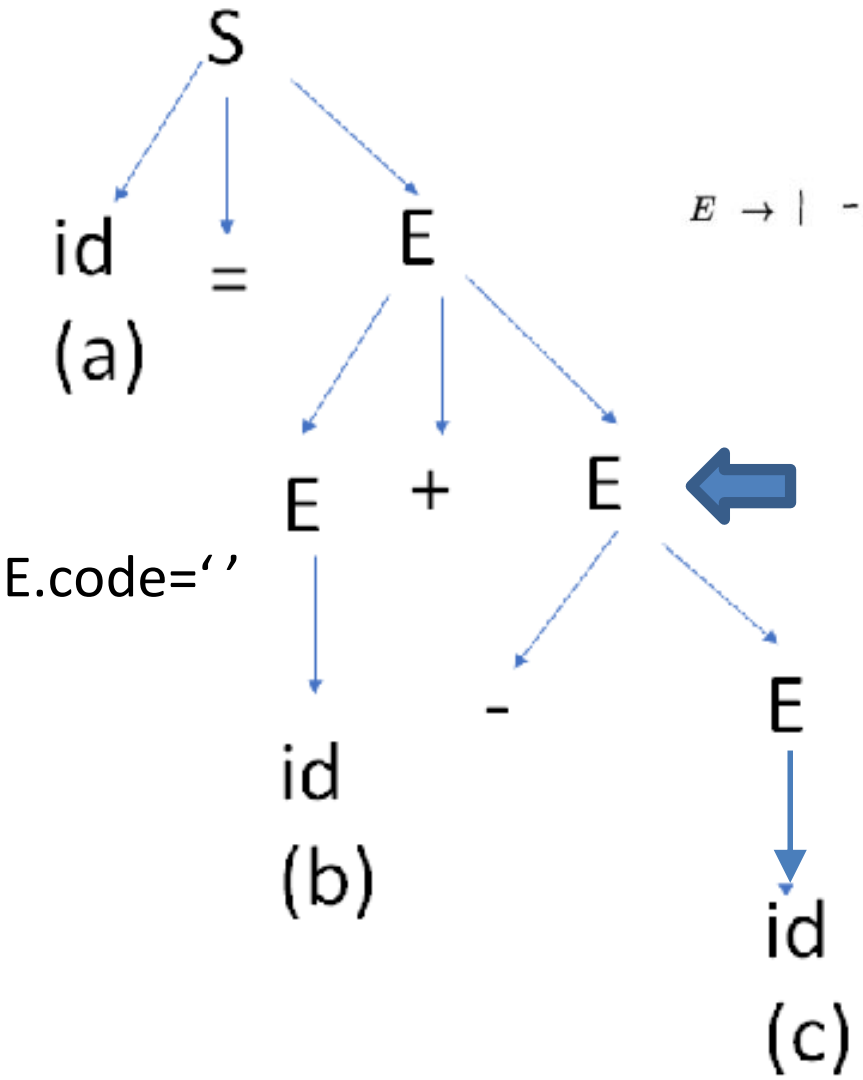
Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ -E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid$ $gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

statement `a = b + - c`



$E \rightarrow \mid \textbf{id}$	$E.addr = top.get(\textbf{id}.lexeme)$ $E.code = ''$

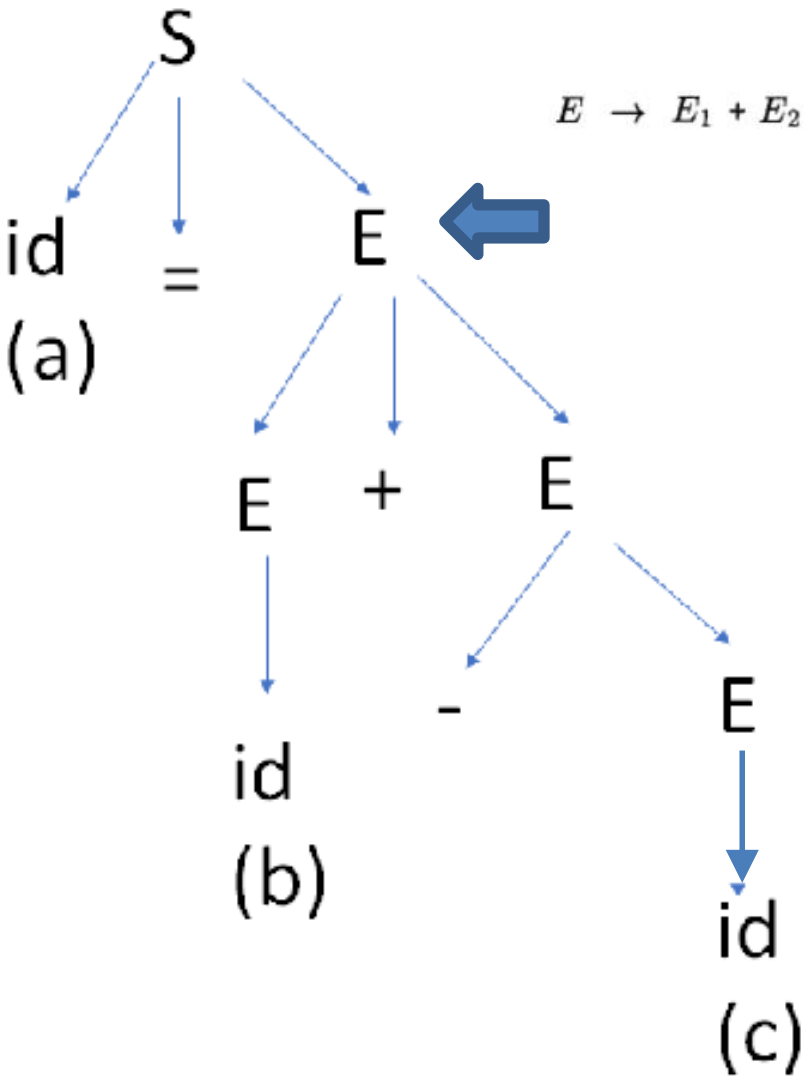


$E \rightarrow | \; - E_1$

$E.addr = \textbf{new}\ Temp()$
$E.code = E_1.code \; ||$
$gen(E.addr\ '='\ '\textbf{minus}'\ E_1.addr)$

E.code=

t_1 = minus c

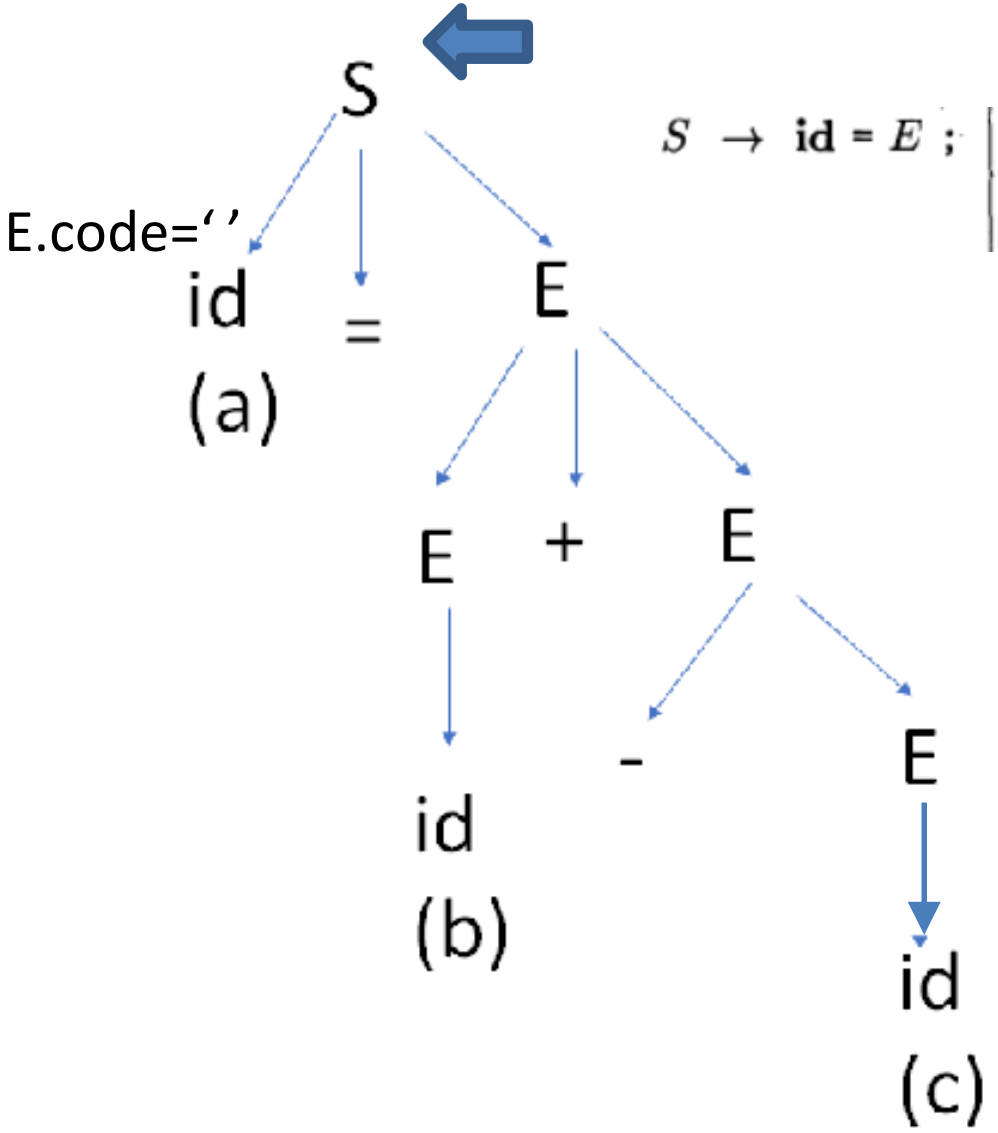


$$E \rightarrow E_1 + E_2 \quad \Big| \quad \begin{array}{l} E.addr = \textbf{new}\ Temp() \\ E.code = E_1.code \mid\mid E_2.code \mid\mid \\ \qquad gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr) \end{array}$$

E.code=

t_1 = minus c

t_2 = b + t_1



$$S \rightarrow \mathbf{id} = E \; ; \quad \Big| \quad S.code = E.code \; || \; gen(top.get(\mathbf{id}.lexeme) \; '=' \; E.addr)$$

S.code=

t_1 = minus c

t_2 = b + t_1

a = t_2

Translation of Expressions

Three-address code for an assignment statement

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E\ ;$	$S.code = E.code \mid\mid$ $gen(top.get(\mathbf{id}.lexeme)\ '='\ E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid E_2.code \mid\mid$ $gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$
$\mid\ -E_1$	$E.addr = \mathbf{new}\ Temp()$ $E.code = E_1.code \mid\mid$ $gen(E.addr\ '='\ '\mathbf{minus}'\ E_1.addr)$
$\mid\ (\ E_1\ )$	$E.addr = E_1.addr$ $E.code = E_1.code$
$\mid\ \mathbf{id}$	$E.addr = top.get(\mathbf{id}.lexeme)$ $E.code = ''$

statement `a = b + - c`

t_1 = minus c

t_2 = b + t_1

a = t_2

Incremental Translation

- So far, ***E.Code*** attributes were long strings
  - Generated incrementally
- In incremental translation, generate only the **new three-address instructions**
- **Past sequence** may either be **retained in memory** for further processing, or it may be **output incrementally**.
- In the incremental approach, gen() not only constructs a three-address instruction,
  - it **appends** the instruction to the sequence of instructions generated so far.

Incremental Translation

$$
\begin{array}{lll}
S \rightarrow \textbf{id} = E \; ; & \{ \; gen(\; top.get(\textbf{id}.lexeme) \; '=' \; E.addr); \; \} \\
E \rightarrow E_1 + E_2 & \{ \; E.addr = \textbf{new} \; Temp\,(); \\
 & \quad gen(E.addr \; '=' \; E_1.addr \; '+' \; E_2.addr); \; \} \\
\quad | \; - E_1 & \{ \; E.addr = \textbf{new} \; Temp\,(); \\
 & \quad gen(E.addr \; '=' \; '\textbf{minus}' \; E_1.addr); \; \} \\
\quad | \; (\; E_1 \;) & \{ \; E.addr = E_1.addr; \; \} \\
\quad | \; \textbf{id} & \{ \; E.addr = top.get(\textbf{id}.lexeme); \; \}
\end{array}
$$

- This **translation scheme generates the same code** as the previous syntax directed definition.
- With the incremental approach, the **E.code attribute is not used**,
    - Since there is a **single sequence of instructions** that is created by **successive calls to gen()**.

Translation of Array Expressions

```
int num[3][4] = {
  {1, 2,  3,  4},
  {5, 6,  7,  8},
  {9, 10, 11, 12}
};
```

8. Indexed copy instructions of the form $x = y[i]$ and $x[i] = y$. The instruction $x = y[i]$ sets x to the value in the location i memory units beyond location y. The instruction $x[i] = y$ sets the contents of the location i units beyond x to the value of y.

Translation of Array Expressions

```
int num[3][4] = {
  {1, 2,  3,  4},
  {5, 6,  7,  8},
  {9, 10, 11, 12}
};
```

Translate 2D index to 1D index

row-wise memory allocation

	<—— row 0 ——>				<—— row 1 ——>				<—— row 2 ——>			
value	1	2	3	4	5	6	7	8	9	10	11	12
address	1000	1002	1004	1006	1008	1010	1012	1014	1016	1018	1020	1022

↑ first element of the array num

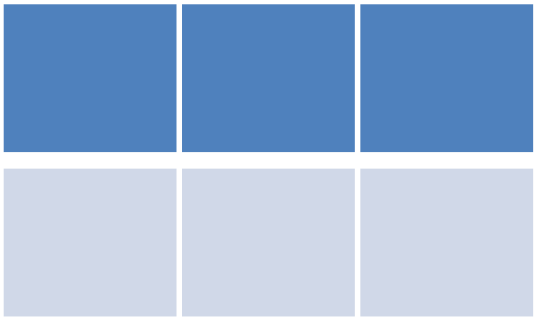
Let w1 be the length of a row (# of columns) and w2 be the size of an element in a row.

The relative address of A[i1][i2] can be calculated by the formula

$$base + i_1 \times w_1 + i_2 \times w_2$$

Translation of Array Expressions

Objective



a is a 2×3 array

expression c + a[i][j]

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a [ t3 ]
t5 = c + t4
```

Translation of Array Expressions

$$
\begin{array}{lll}
S \rightarrow \textbf{id} = E\ ; & \{\ gen(\ top.get(\textbf{id}.lexeme)\ '='\ E.addr);\ \} \\
\quad \mid\ L = E\ ; & \{\ gen(L.addr.base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \} \\
E \rightarrow E_1 + E_2 & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \} \\
\quad \mid\ \textbf{id} & \{\ E.addr = top.get(\textbf{id}.lexeme);\ \} \\
\quad \mid\ L & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \} \\
L \rightarrow \textbf{id}\ [\ E\ ] & \{\ L.array = top.get(\textbf{id}.lexeme);\ L.type = L.array.type.elem;\ L.addr = \textbf{new}\ Temp();\ gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \} \\
\quad \mid\ L_1\ [\ E\ ] & \{\ L.array = L_1.array;\ L.type = L_1.type.elem;\ t = \textbf{new}\ Temp();\ L.addr = \textbf{new}\ Temp();\ gen(t\ '='\ E.addr\ '*'\ L.type.width);\ gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \}
\end{array}
$$

L.addr denotes a temporary variable containing the **offset** of the **array elements** (array index)

$$base + i_1 \times w_1 + i_2 \times w_2$$

Translation of Array Expressions

$$
\begin{array}{lll}
S \rightarrow \textbf{id} = E\ ; & \{\ gen(\ top.get(\textbf{id}.lexeme)\ '='\ E.addr);\ \} \\
\quad | \quad L = E\ ; & \{\ gen(L.addr.base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \} \\
E \rightarrow E_1 + E_2 & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \} \\
\quad | \quad \textbf{id} & \{\ E.addr = top.get(\textbf{id}.lexeme);\ \} \\
\quad | \quad L & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \} \\
L \rightarrow \textbf{id}\ [\ E\ ] & \{\ L.array = top.get(\textbf{id}.lexeme);\ L.type = L.array.type.elem;\ L.addr = \textbf{new}\ Temp();\ gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \} \\
\quad | \quad L_1\ [\ E\ ] & \{\ L.array = L_1.array;\ L.type = L_1.type.elem;\ t = \textbf{new}\ Temp();\ L.addr = \textbf{new}\ Temp();\ gen(t\ '='\ E.addr\ '*'\ L.type.width);\ \}\ gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \}
\end{array}
$$

- **L.array** is a pointer to the symbol-table entry for the array name.
- **L.array.base** indicates base address of the array -- array name
- **L.type** is the type t of the subarray generated by L.
- For **array type t**, we assume that its width is given by **t.width**.
- For any **array type t**, **t.elem** gives the array element.

Translation of Array Expressions

$$
\begin{array}{lll}
S \rightarrow \textbf{id} = E\ ; & \{\ gen(\ top.get(\textbf{id}.lexeme)\ '='\ E.addr);\ \} & \\
\quad \mid\ L = E\ ; & \{\ gen(L.addr.base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \} & \\
E \rightarrow E_1 + E_2 & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \} & \\
\quad \mid\ \textbf{id} & \{\ E.addr = top.get(\textbf{id}.lexeme);\ \} & \\
\quad \mid\ L & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \} & \\
L \rightarrow \textbf{id}\ [\ E\ ] & \{\ L.array = top.get(\textbf{id}.lexeme);\ L.type = L.array.type.elem;\ L.addr = \textbf{new}\ Temp();\ gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \} & \\
\quad \mid\ L_1\ [\ E\ ] & \{\ L.array = L_1.array;\ L.type = L_1.type.elem;\ t = \textbf{new}\ Temp();\ L.addr = \textbf{new}\ Temp();\ gen(t\ '='\ E.addr\ '*'\ L.type.width);\ gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \} &
\end{array}
$$

Standard

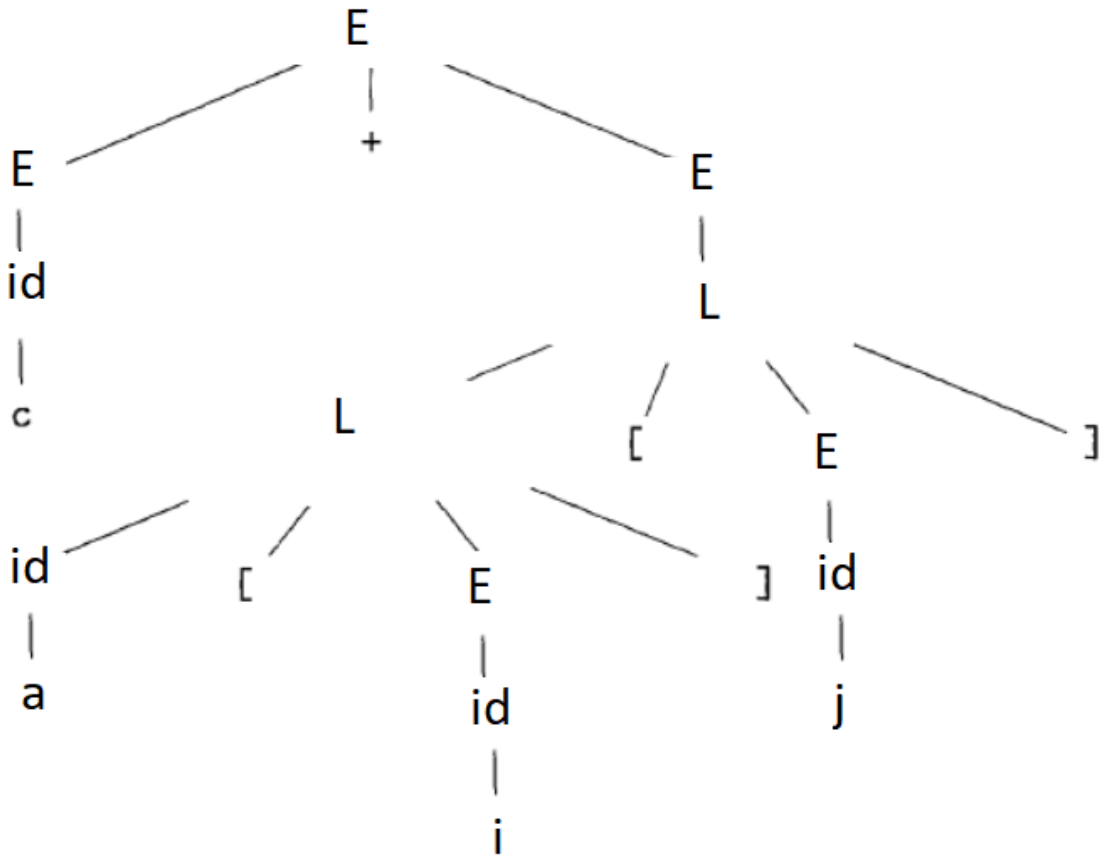
Standard

Translation of Array Expressions

$$
\begin{array}{lll}
S \rightarrow \textbf{id} = E\ ; & \{\ gen(\ top.get(\textbf{id}.lexeme)\ '='\ E.addr);\ \} \\
\quad | \quad L = E\ ; & \{\ gen(L.addr.base\ '['\ L.addr\ ']'\ '='\ E.addr);\ \} \\
E \rightarrow E_1 + E_2 & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);\ \} \\
\quad | \quad \textbf{id} & \{\ E.addr = top.get(\textbf{id}.lexeme);\ \} \\
\quad | \quad L & \{\ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']');\ \} \\
L \rightarrow \textbf{id}\ [\ E\ ] & \{\ L.array = top.get(\textbf{id}.lexeme);\ L.type = L.array.type.elem;\ L.addr = \textbf{new}\ Temp();\ gen(L.addr\ '='\ E.addr\ '*'\ L.type.width);\ \} \\
\quad | \quad L_1\ [\ E\ ] & \{\ L.array = L_1.array;\ L.type = L_1.type.elem;\ t = \textbf{new}\ Temp();\ L.addr = \textbf{new}\ Temp();\ gen(t\ '='\ E.addr\ '*'\ L.type.width);\ gen(L.addr\ '='\ L_1.addr\ '+'\ t);\ \}
\end{array}
$$

Translation of Array Expressions

expression c + a[i][j]



$S \rightarrow \mathbf{id} = E \;;$

$\mid \; L = E \;;$

$E \rightarrow E_1 + E_2$

$\mid \; \mathbf{id}$

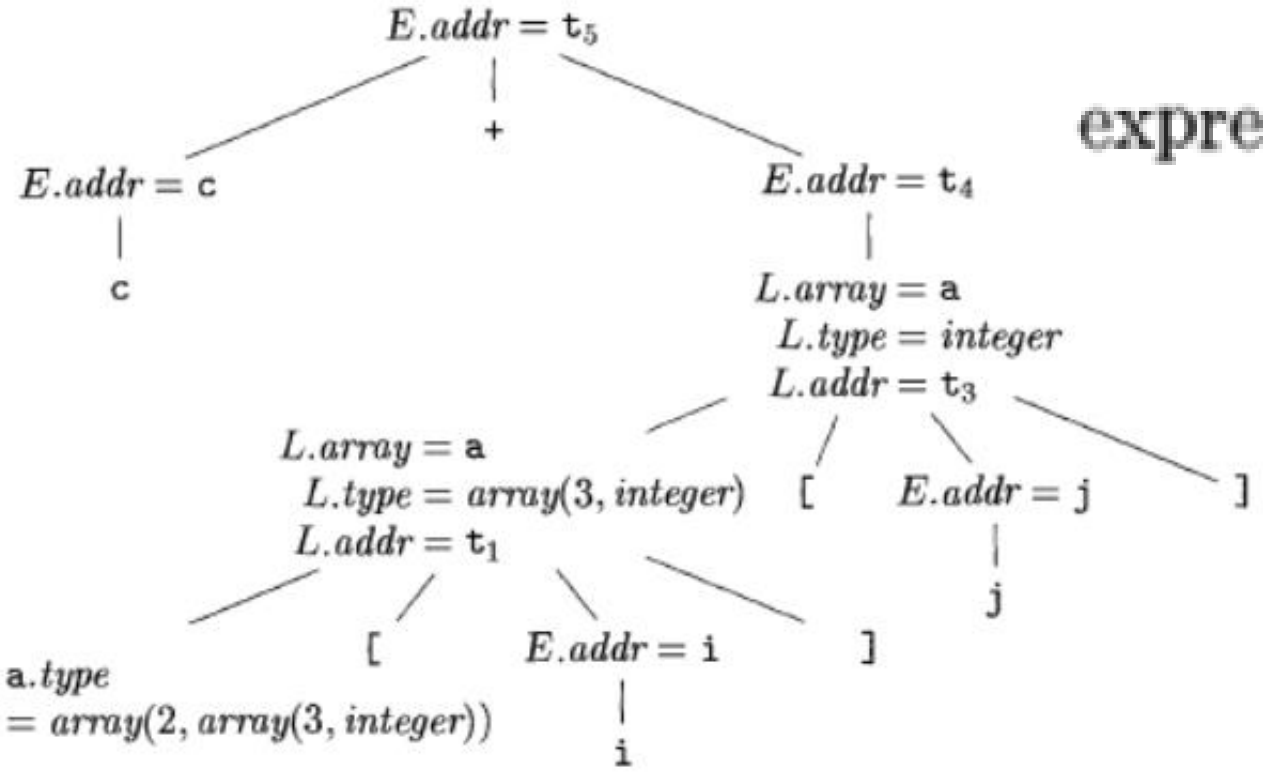
$\mid \; L$

$L \rightarrow \mathbf{id} \; [\; E \;]$

$\mid \; L_1 \; [\; E \;]$

Parse tree

Translation of Array Expressions



expression c + a[i][j]

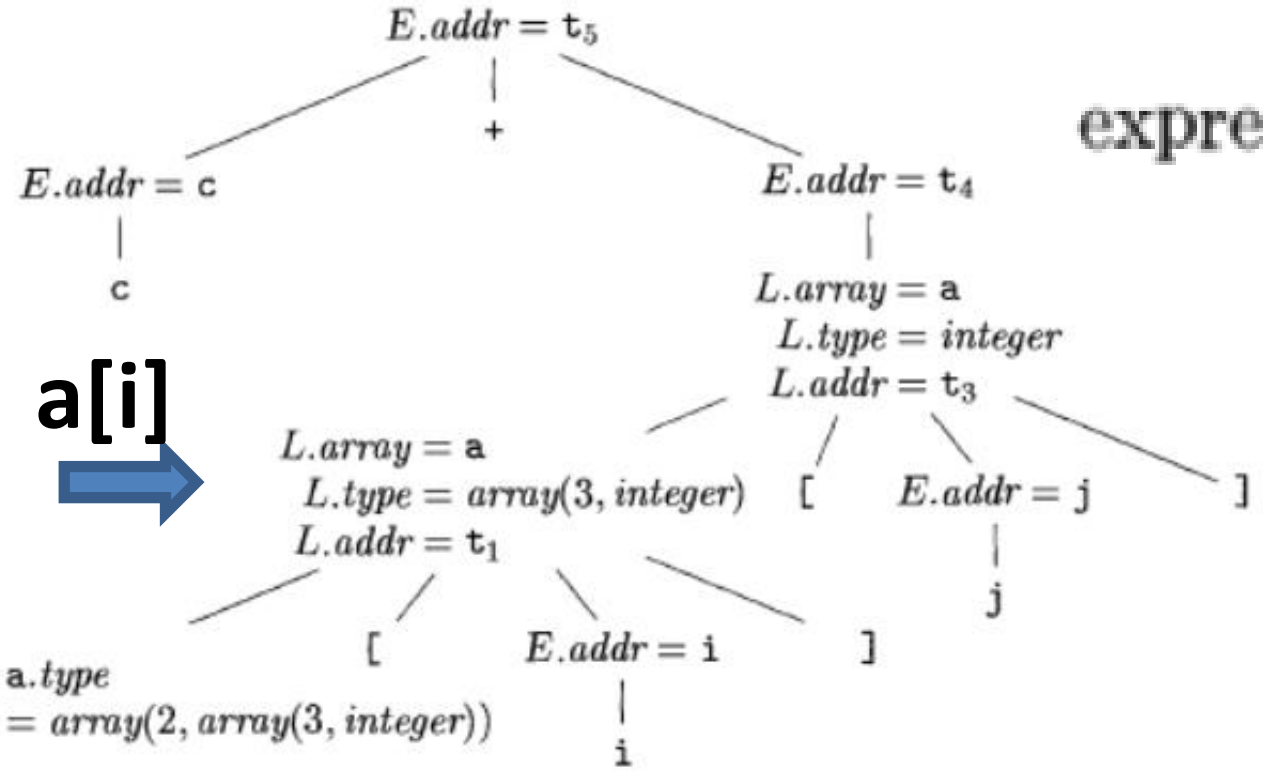
$E \rightarrow L$ { $E.addr =$ **new** $Temp()$;
$gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']')$; }

$E \rightarrow$ **id** { $E.addr = top.get(\textbf{id}.lexeme)$; }

$L \rightarrow$ **id** [E] { $L.array = top.get(\textbf{id}.lexeme)$;
$L.type = L.array.type.elem$;
$L.addr =$ **new** $Temp()$;
$gen(L.addr\ '='\ E.addr\ '*'\ L.type.width)$; }

| L_1 [E] { $L.array = L_1.array$;
$L.type = L_1.type.elem$;
$t =$ **new** $Temp()$;
$L.addr =$ **new** $Temp()$;
$gen(t\ '='\ E.addr\ '*'\ L.type.width)$; }
$gen(L.addr\ '='\ L_1.addr\ '+'\ t)$; }

Translation of Array Expressions



expression c + a[i][j]

a[i]

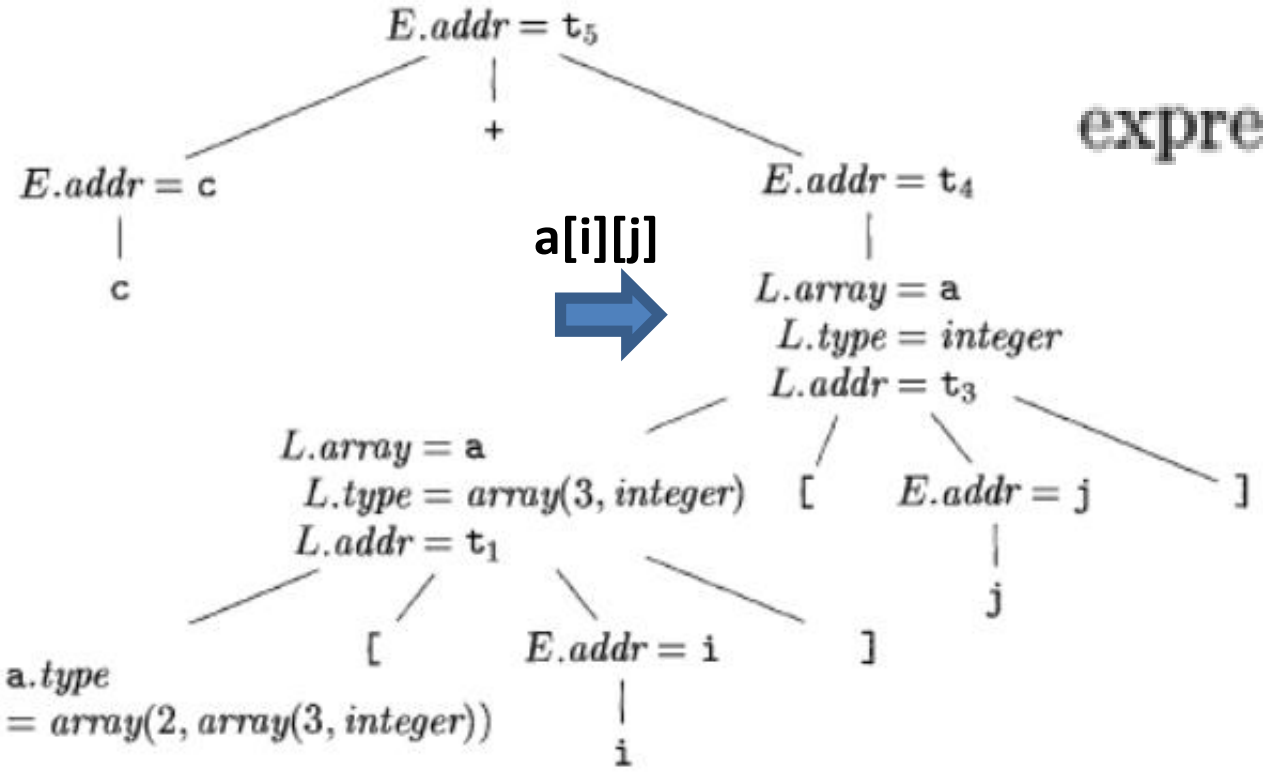
t_1 = i * 12

$L \rightarrow$ **id** [E] { $L.array = top.get(\textbf{id}.lexeme)$;
$L.type = L.array.type.elem$;
$L.addr =$ **new** $Temp()$;
$gen(L.addr\ '='\ E.addr\ '*'\ L.type.width)$; }

| L_1 [E] { $L.array = L_1.array$;
$L.type = L_1.type.elem$;
$t =$ **new** $Temp()$;
$L.addr =$ **new** $Temp()$;
$gen(t\ '='\ E.addr\ '*'\ L.type.width)$; }
$gen(L.addr\ '='\ L_1.addr\ '+'\ t)$; }

$E \rightarrow$ **id** { $E.addr = top.get(\textbf{id}.lexeme)$; }

Translation of Array Expressions



expression c + a[i][j]

$t_1 = i * 12$

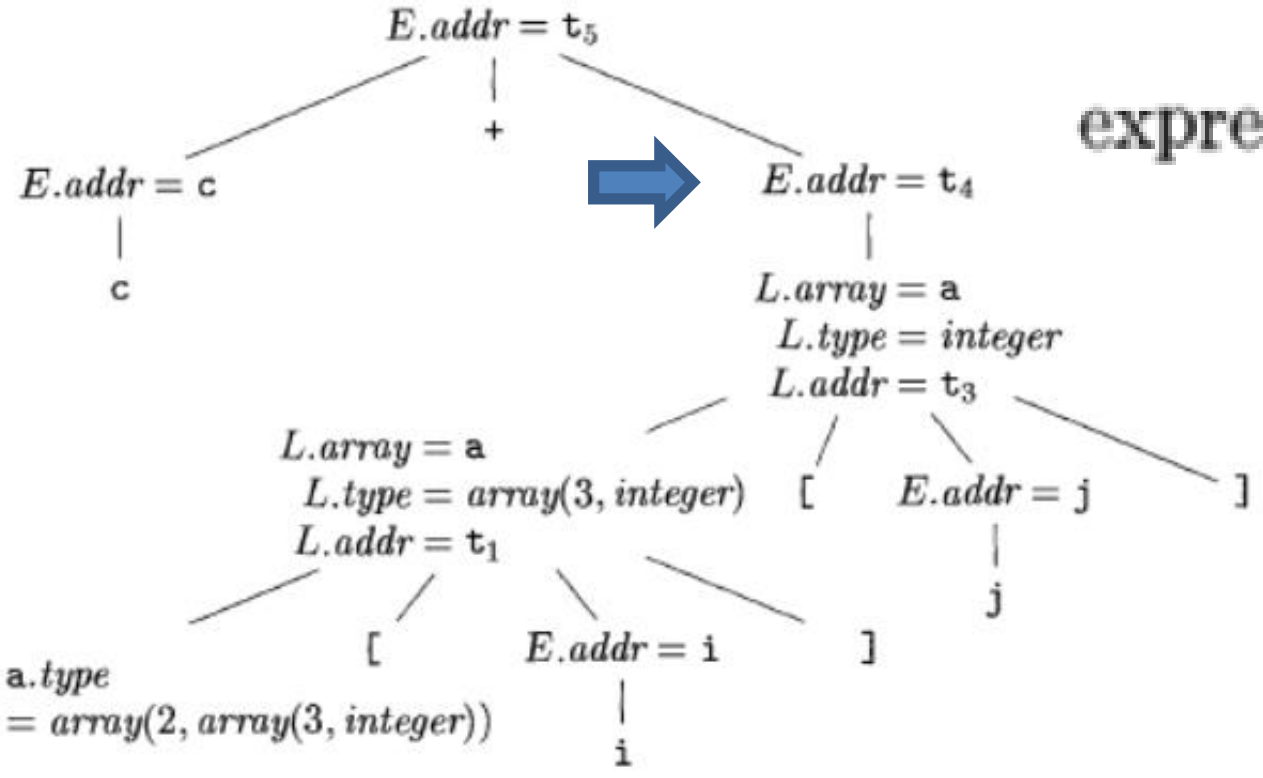
$t_2 = j * 4$
$t_3 = t_1 + t_2$

$L \rightarrow \textbf{id}\ [\ E\ ]$ { $L.array = top.get(\textbf{id}.lexeme)$;
$L.type = L.array.type.elem$;
$L.addr = \textbf{new}\ Temp()$;
$gen(L.addr\ '='\ E.addr\ '*'\ L.type.width)$; }

| $L_1\ [\ E\ ]$ { $L.array = L_1.array$;
$L.type = L_1.type.elem$;
$t = \textbf{new}\ Temp()$;
$L.addr = \textbf{new}\ Temp()$;
$gen(t\ '='\ E.addr\ '*'\ L.type.width)$; }
$gen(L.addr\ '='\ L_1.addr\ '+'\ t)$; }

$E \rightarrow \textbf{id}$ { $E.addr = top.get(\textbf{id}.lexeme)$; }

Translation of Array Expressions



expression c + a[i][j]

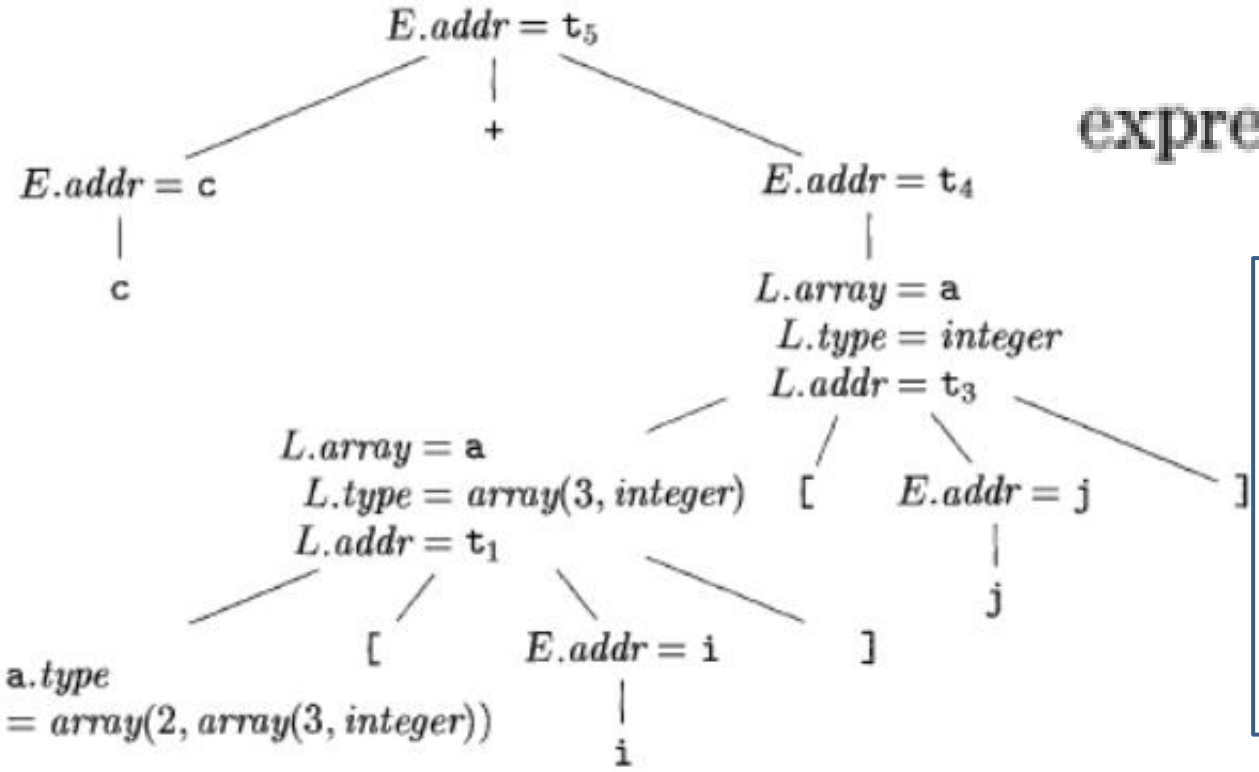
$$t_1 = i * 12$$

$$t_2 = j * 4$$
$$t_3 = t_1 + t_2$$

$$t_4 = a [t_3]$$

$$E \rightarrow L \qquad \{ E.addr = \textbf{new}\ Temp();\ gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']'); \}$$

Translation of Array Expressions



expression c + a[i][j]

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a [ t3 ]
t5 = c + t4
```

$E \rightarrow L$ { $E.addr =$ **new** $Temp()$;
$gen(E.addr\ '='\ L.array.base\ '['\ L.addr\ ']')$; }

$E \rightarrow$ **id** { $E.addr = top.get(\textbf{id}.lexeme)$; }

$L \rightarrow$ **id** [E] { $L.array = top.get(\textbf{id}.lexeme)$;
$L.type = L.array.type.elem$;
$L.addr =$ **new** $Temp()$;
$gen(L.addr\ '='\ E.addr\ '*'\ L.type.width)$; }

| L_1 [E] { $L.array = L_1.array$;
$L.type = L_1.type.elem$;
$t =$ **new** $Temp()$;
$L.addr =$ **new** $Temp()$;
$gen(t\ '='\ E.addr\ '*'\ L.type.width)$; }
$gen(L.addr\ '='\ L_1.addr\ '+'\ t)$; }